

21 Most Asked Playwright Interview Questions for QA Automation Engineers

Question 1: Why did you choose Playwright over Selenium?

Answer

We chose **Playwright** because it is faster and provides many built-in features. It has **Auto-Wait**, which automatically waits for elements before performing actions, reducing flaky tests. Playwright also supports **multiple browsers**, **parallel execution**, **API testing**, and **HTML reports** without using many third-party libraries. These features make our automation framework more stable and easier to maintain.

Important Points

- Faster execution
- Auto-Wait
- Parallel Execution
- Multi-browser support
- API Testing
- HTML Reports
- Less third-party dependency
- Stable automation

Question 2: What is Auto-Wait in Playwright?

Answer

Auto-Wait is a built-in feature in Playwright. Before performing actions like **click()**, **fill()**, or **hover()**, Playwright automatically waits until the element is **visible, enabled, stable, and ready for interaction**. This reduces test failures and removes the need for unnecessary explicit waits.

Important Points

- Built-in feature
- Waits automatically
- Checks element is visible
- Checks element is enabled
- Checks element is stable
- Reduces flaky tests
- Less use of **waitForTimeout()**

Question 3: What is a Flaky Test?

Answer

A **Flaky Test** is a test that **sometimes passes and sometimes fails without any changes in the code**. This usually happens due to synchronization issues, unstable locators, network delays, hard waits, or dynamic elements. Playwright's **Auto-Wait** helps reduce flaky tests.

Important Points

- Passes sometimes, fails sometimes
- No code changes
- Synchronization issues
- Dynamic elements
- Network delay
- Hard waits
- Unstable locators
- Auto-Wait reduces flaky tests

Question 4: What are Browser, BrowserContext, and Page in Playwright?

Answer

- **Browser** is the browser instance (Chromium, Firefox, or WebKit).
- **BrowserContext** is an isolated browser session with its own cookies, cache, and local storage.
- **Page** represents a single browser tab where we perform automation actions like **click()**, **fill()**, and **goto()**.

Important Points

- Browser = Browser instance
- BrowserContext = Isolated session
- Page = Browser tab
- Multiple contexts can run in one browser
- Each context has separate cookies and storage
- Used for multi-user testing and parallel execution

Question 5: Why do we use BrowserContext instead of opening multiple browsers?

Answer

We use **BrowserContext** because it creates **multiple isolated sessions within the same browser**. Each context has its own cookies, cache, and local storage, so user data is not shared.

It is faster, uses fewer system resources, and is ideal for parallel and multi-user testing.

Important Points

- `BrowserContext` = Isolated session
- No sharing of cookies or local storage
- Faster than opening multiple browsers
- Consumes less memory
- Best for parallel execution
- Used for multi-user testing (Admin/User)

Question 6: What is the difference between `browser.newPage()` and `browser.newContext().newPage()`?

Answer

`browser.newPage()` is a shortcut that creates a new page with a new browser context automatically. It is suitable for simple scripts.

`browser.newContext().newPage()` first creates a new browser context and then opens a page. This provides better session isolation and is the preferred approach in real-time automation frameworks.

Important Points

- `browser.newPage()` = Shortcut method
- `browser.newContext().newPage()` = Creates context first, then page
- `BrowserContext` provides session isolation
- No sharing of cookies, cache, or local storage

- Preferred in real-time projects
- Supports parallel execution and multi-user testing

Question 7: What is Page Object Model (POM)?

Answer

Page Object Model (POM) is a design pattern where locators and page actions are stored in separate page classes, and test cases are written separately. This makes the framework easy to maintain, reusable, and reduces code duplication.

Important Points

- Separates locators from test scripts
- Improves code reusability
- Easy to maintain
- Reduces code duplication
- Better readability
- Widely used in real-time projects

Question 8: Why do we use Fixtures in Playwright?

Answer

Fixtures are used to share common setup across multiple test cases. Instead of writing the same code repeatedly, we write it once in a fixture and reuse it wherever needed. They help initialize resources like Page, Browser, BrowserContext, Login, and Test Data before test execution. This reduces duplicate code, improves readability, and makes the framework easier to maintain.

Key Points to Remember

- Reusable setup
- Reduces code duplication
- Easy to maintain
- Provides Page, Browser, BrowserContext
- Supports custom fixtures
- Improves test readability
- Used in real-time frameworks

Question 9: What are Hooks in Playwright?

Answer

Hooks are used to execute common code before or after test execution. Instead of writing the same setup or cleanup code in every test, we write it once in a hook. Hooks help with tasks like launching the application, logging in, creating test data, and closing the browser, making the test suite clean and easy to maintain.

Key Points to Remember

- Runs code before or after tests
- Avoids duplicate code
- Used for setup and cleanup
- Keeps tests clean and organized
- Easy to maintain
- Hooks: `beforeAll`, `beforeEach`, `afterEach`, `afterAll`

Question 10: What is the difference between `beforeAll()` and `beforeEach()` in Playwright?

Answer

`beforeAll()` runs only once before all the test cases in a test file. It is used for common setup that is needed only one time.

`beforeEach()` runs before every test case. It is used when each test needs the same setup, such as logging in or navigating to a page.

Key Points to Remember

- `beforeAll()` → Runs once
- `beforeEach()` → Runs before every test
- Used for common setup
- Reduces duplicate code
- Improves test maintenance
- Common use: Login, Launch application, Test data setup

Question 11: What is the difference between `locator()` and `page.$()` in Playwright?

Answer

`locator()` is the recommended way to find elements in Playwright. It supports Auto-Wait and automatically retries until the element is ready.

`page.$()` returns the element immediately. It does not support Auto-Wait, so it may fail if the element is not available. That's why `locator()` is preferred in real-time projects.

Key Points to Remember

- `locator()` is recommended
- Supports Auto-Wait
- Automatically retries
- `page.$()` does not support Auto-Wait
- `locator()` is more reliable
- Used in real-time frameworks

Question 12: What are the different types of waits in Playwright?

Answer

Playwright provides different waiting mechanisms to handle dynamic web applications and ensure elements are ready before interaction.

① Auto Wait (Recommended)

Playwright automatically waits for elements to become visible, enabled, and actionable before performing actions.

Example:

```
await page.locator('#loginBtn').click();
```

No explicit wait is needed because Playwright automatically waits.

② Explicit Wait

Used when we need to wait for a specific condition.

Examples:

```
await page.waitForSelector('#username');  
  
await page.waitForURL('**/dashboard');  
  
await page.waitForLoadState('networkidle');  
  
await page.waitForResponse('**/users');
```

Used when waiting for elements, page navigation, page load, or API responses.

③ Hard Wait (Not Recommended)

Used to pause execution for a fixed amount of time.

Example:

```
await page.waitForTimeout(5000);
```

This waits for exactly 5 seconds regardless of whether the application is ready.

- ✗ Slows down execution.
 - ✗ Can create flaky tests.
 - ✗ Avoid in real projects unless absolutely necessary.
-

Interview Tip

"In Playwright, I primarily rely on Auto-Wait because it makes tests more stable and reduces flakiness. When needed, I use Explicit Waits such as `waitForSelector()`, `waitForURL()`, or `waitForResponse()`. I generally avoid Hard Waits (`waitForTimeout()`) because they increase execution time and make tests less reliable."

Question 13: What is the difference between ElementHandle and Locator in Playwright?

Answer

An **ElementHandle** is a reference to a specific DOM element. Once Playwright finds the element, it stores that reference. If the page is refreshed, updated, or the element is recreated, the reference becomes stale (it points to the old element, which no longer exists on the page). This can cause the test to fail.

A **Locator** is a smart way to find an element. It searches for the element every time an action like `click()` or `fill()` is performed. It supports Auto-Wait and always interacts with the latest available element. This makes Locator more reliable and is the recommended approach in Playwright.

Key Points to Remember

- **ElementHandle** → Stores a reference to one specific element
- **Locator** → Finds the element every time an action is performed
- **ElementHandle** can become stale (points to an old element after the page changes)
- **Locator** always works with the latest element
- **ElementHandle** does not support Auto-Wait
- **Locator** supports Auto-Wait
- **Locator** is recommended for real-time projects

Easy Example

Suppose you click the Refresh button, and the page recreates the Submit button.

- **ElementHandle**: Still points to the old Submit button, which no longer exists, so the test may fail.
- **Locator**: Searches again and finds the new Submit button, so the test continues successfully.

Question 14: What are the different Locator strategies in Playwright?

Answer

Playwright provides different locator strategies to identify web elements. We should always use the most reliable locator. In real-time projects, `getByRole()`, `getByLabel()`, and `getByTestId()` are preferred because they are stable and easy to maintain. If these are not available, we can use CSS Selector or XPath.

Key Points to Remember

- **getByRole()** → Locate by role (Button, Link, Checkbox)
- **getByText()** → Locate by visible text
- **getByLabel()** → Locate input fields using labels
- **getByPlaceholder()** → Locate using placeholder text
- **getByAltText()** → Locate images by alt text
- **getTitle()** → Locate using title attribute
- **getByTestId()** → Locate using test ID (Most stable)
- **CSS Selector** → Used when Playwright locators are not available
- **XPath** → Used for complex or dynamic elements

Interview Tip:

Preferred order of locators:

1. **getByRole()**
2. **getByLabel()**
3. **getByTestId()**

4. `getByText()`
5. CSS Selector
6. XPath (Last preference)

Question 15: What is the difference between CSS Selector and XPath?

Answer

CSS Selector is used to locate elements based on HTML attributes, class, ID, or tag name. It is generally faster and easier to write.

XPath is used to locate elements based on the HTML structure and relationships between elements, such as parent, child, or sibling. It is useful for handling dynamic and complex elements when CSS is not sufficient.

Key Points to Remember

- CSS Selector → Faster
- XPath → More flexible
- CSS uses ID, Class, Tag, Attributes
- XPath uses Parent, Child, Sibling relationships
- CSS cannot traverse backward (parent)
- XPath can navigate both forward and backward
- Use CSS whenever possible
- Use XPath for complex or dynamic elements

Question 16: What is the difference between `getByRole()` and `getByText()` in Playwright?

Answer

`getByRole()` locates elements based on their role and accessible name, such as button, link, or textbox. It is more reliable and is the recommended locator in Playwright.

`getByText()` locates elements using their visible text. It is useful when the text is unique, but if multiple elements have the same text, it may return multiple matches.

Key Points to Remember

- `getByRole()` → Uses role and accessible name
- `getByText()` → Uses visible text
- `getByRole()` is more reliable
- Recommended by Playwright
- `getByText()` is useful for unique text
- Prefer `getByRole()` whenever possible

Question 17: What are Assertions in Playwright?

Answer

Assertions are used to verify whether the actual result matches the expected result. They help confirm that the application is working correctly. If the expected condition is met, the test passes; otherwise, the test fails. Playwright provides built-in assertions using the `expect()` method.

Key Points to Remember

- Used to validate test results
- Compares actual and expected values
- Uses `expect()` method

- If validation passes → Test passes
- If validation fails → Test fails
- Supports Auto-Retry for web elements

Question 18: What is the difference between Hard Assert and Soft Assert?

Answer

A Hard Assert stops the test execution immediately if an assertion fails. The remaining test steps will not be executed.

A Soft Assert continues executing the remaining test steps even if an assertion fails. At the end of the test, it reports all assertion failures together.

Key Points to Remember

- Hard Assert → Stops execution on failure
- Soft Assert → Continues execution after failure
- Hard Assert → Reports first failure
- Soft Assert → Reports all failures at the end
- Playwright uses Hard Assert by default
- Soft Assert: `expect.soft()`

Question 19: What is playwright.config.ts? Why do we use it?

Answer

`playwright.config.ts` is the main configuration file in Playwright. It is used to manage common settings for all test

cases, such as browser configuration, base URL, timeouts, retries, parallel execution, reporters, screenshots, videos, and projects. This avoids repeating the same configuration in every test.

Key Points to Remember

- Main configuration file
 - Stores common test settings
 - Configure browser and baseURL
 - Set timeout and retries
 - Enable parallel execution
 - Configure reports, screenshots, and videos
 - Supports multiple browser projects
-

Question 20: What is Parallel Execution in Playwright?

Answer

Parallel execution means running multiple test cases at the same time instead of one after another. This reduces the overall execution time and helps complete the test suite faster. Playwright supports parallel execution by default, making it suitable for large automation projects.

Key Points to Remember

- Runs multiple tests simultaneously
- Reduces execution time
- Faster than sequential execution
- Improves test efficiency
- Supported by Playwright
- Best for large test suites

Question 21: What is Trace Viewer in Playwright?

Answer

Trace Viewer is a built-in debugging tool in Playwright. It records every step of the test execution, including actions, screenshots, network requests, console logs, and errors. It helps identify the exact step where the test failed and makes debugging easier.

Key Points to Remember

- Built-in debugging tool
- Records every test step
- Captures screenshots
- Shows network requests
- Displays console logs and errors
- Helps analyze failed tests
- Makes debugging easier

